



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Universidad Nacional de Colombia - sede Bogotá
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial
Curso: Ingeniería de software I

Estudiantes: Camilo Andres Lievano Rendon, Juan Mateo Rozo Torres, Juan Diego Lasso Montaña, Rene Alejandro Barrero Gonzalez.



Patrones de Comportamiento



Command

Descripción general:

El patrón *Command* encapsula una solicitud como un objeto, permitiendo parametrizar acciones, registrar historial, deshacer o ejecutar tareas de manera flexible.

Aplicación en ArchiBox:

Cada acción que el usuario realiza —como registrar un libro, asignar una tarea, cambiar el estado de un libro o clasificarlo— puede representarse como un comando. Esto facilita:

- La creación de un historial de auditoría detallado.
- La reutilización de acciones.
- La ejecución o repetición automática de ciertas operaciones.

Ejemplo:

Una clase `RegistrarLibroCommand` puede tener un método `execute()` que registra el libro y genera automáticamente una entrada en la tabla de historial.

Justificación:

Permite separar la lógica de ejecución de las acciones de la interfaz y facilita la trazabilidad requerida por el sistema.



Observer

Descripción general:

El patrón *Observer* permite que uno o más objetos se actualicen automáticamente cuando el estado de otro objeto cambia, manteniendo una sincronización eficiente.

Aplicación en ArchiBox:

- Cuando un usuario selecciona un libro en la lista principal, el panel de detalle se actualiza automáticamente para mostrar la información correspondiente.
- Cuando se ejecuta una acción (por ejemplo, completar una etapa del flujo), se genera una nueva entrada en el sistema de auditoría automáticamente.

Implementación técnica:

Este patrón se implementa mediante señales y slots en PyQt (`QObject` y `pyqtSignal`).

Justificación:

Mejora la experiencia de usuario, mantiene sincronizada la interfaz y reduce el acoplamiento entre componentes visuales.

Strategy

Descripción general:

El patrón *Strategy* permite seleccionar algoritmos o comportamientos en tiempo de ejecución, encapsulando cada uno en una clase distinta.

Aplicación en ArchiBox:

Se utiliza para implementar distintos criterios de asignación de tareas a usuarios:

- Asignación manual.
- Asignación automática por disponibilidad.
- Asignación por carga de trabajo o rol.

Ejemplo:

Clase base `EstrategiaAsignacion`, con subclases `Manual`, `Automatica`, etc., que comparten una interfaz común.

Justificación:

Permite extender fácilmente la lógica de asignación sin modificar el código existente, cumpliendo con el principio de abierto/cerrado.

Patrones Creacionales

Singleton

El patrón *Singleton* asegura que una clase tenga una única instancia global y proporciona un punto de acceso controlado a ella.

Aplicación en ArchiBox:

- Para manejar la conexión a la base de datos SQLite.
- Para manejar la sesión activa del usuario (información del usuario autenticado durante la ejecución del sistema).

Justificación:

Evita múltiples conexiones simultáneas o sesiones inconsistentes. Garantiza consistencia en recursos compartidos a lo largo de toda la aplicación.

 Factory Method**Descripción general:**

El patrón *Factory Method* permite delegar la creación de objetos a clases especializadas, evitando el uso directo de constructores.

Aplicación en ArchiBox:

- Para crear instancias de tareas según la etapa del libro (*TareaRevision*, *TareaDigitalizacion*, etc.).
- Para generar formularios específicos dependiendo del tipo de acción o rol del usuario (*FormularioClasificacion*, *FormularioRevisión*, etc.).
- Para crear distintos tipos de reportes (*ReportePorUsuario*, *ReportePorEtapa*, *ReporteGeneral*).
-

Justificación:

Este patrón permite agregar nuevas etapas, formularios o reportes sin modificar el código existente, facilitando la escalabilidad del sistema y evitando estructuras condicionales repetitivas.